DATAWISE AI

DataWise AI: Developer Guide

Architecture Reference, Codebase Extension, Agent Design, and API Integration

Document Type:	Engineering & Contributor Guide
Target Audience:	Backend Engineers, Frontend Developers, AI/Agent Engineers
Document Version:	v1.0.0 (Engineering Release)
Date of Issue:	October 27, 2026
Repository Status:	Verified Codebase & Production Architecture
Orchestration:	LangGraph 16-Agent Autonomous Architecture

Architectural Foundation: Deterministic ML execution engine paired with autonomous multi-agent reasoning, strict data leakage boundaries, and human-in-the-loop validation.

1. Development Environment Setup

This guide provides complete instructions for setting up, running, debugging, and extending the DataWise AI platform locally.

Step	Configuration Action	Execution Command
Step	Action	Command / Details
1. Clone & Env	Clone repo and configure environment	cp .env.example .env (configure DB, Redis, LLM keys)
2. Python Virtualenv	Create Python 3.11 virtual environment	python -m venv .venv; source .venv/bin/activate (or .venv\Scripts\activate)
3. Backend Dependencies	Install backend requirements	pip install -r requirements.txt (fastapi, scikit-learn, optuna, etc.)
4. Migrations	Apply SQLite / PostgreSQL migrations	python apply_sqlite_migrations.py (or alembic upgrade head)
5. Frontend Setup	Install Vite frontend packages	cd frontend && npm install
6. Start Backend	Launch FastAPI server with hot-reload	uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
7. Start Frontend	Launch Vite dev server	cd frontend && npm run dev (accessible at http://localhost:5173)

2. Repository Architecture Map

The codebase is strictly organized by separation of concerns:

- `backend/agents/`: Autonomous agent definitions implementing the base agent pattern.
- `backend/graphs/`: LangGraph workflow graphs (master_graph, preprocessing_graph, training_graph, etc.).
- `backend/ml/`: Deterministic data science routines (profiling, preprocessing, training, Optuna tuning).
- `backend/app/models/db_models.py`: 37 SQLAlchemy models covering projects, runs, models, and security.
- `backend/recovery/`: 18-module error resilience, fallback, and circuit-breaker engine.
- `frontend/src/`: React 18 application with pages for Auth, Dashboard, NewProject, and Workbench.

3. Extending the Platform: Adding a New Agent

To add a new specialized agent to DataWise AI:

1. Create `backend/agents/my_custom_agent/agent.py` inheriting from `BaseAgent`.
2. Implement the `async def execute(self, state: DataScienceState) -> DataScienceState` method.
3. Ensure deterministic computations call tools in `backend/ml/` rather than prompting the LLM directly.
4. Register the new agent node and transitions in `backend/graphs/master_graph.py`.
5. Add unit tests in `backend/tests/test_agents/` verifying state updates and error handling.

4. Adding a New Machine Learning Algorithm

To introduce a new estimator (e.g., CatBoost, Support Vector Machines):

- Define the hyperparameter search space in `backend/ml/tuning/hyperparameter_spaces.py`.
- Register model factory and training logic in `backend/ml/models/model_factory.py`.
- Update `MLStrategyAgent` in `backend/agents/10_ml_strategy_agent/` to include selection rules.